Implementação 3
Caminho mínimo com o menor número de arestas

1. Objetivo

O objetivo deste trabalho foi implementar uma busca de caminho mínimo em grafos direcionados e ponderados, considerando apenas pesos positivos. Além de encontrar o menor peso total entre a origem e o destino, o programa deve escolher o caminho com menos arestas quando houver empate no peso.

2. Implementação

A solução foi desenvolvida em Java, usando listas de adjacência para armazenar o grafo e o algoritmo de Dijkstra para realizar a busca. Para cada vértice, são guardados a menor distância encontrada, o número de arestas desse percurso e o vértice anterior, que permite reconstruir o caminho ao final.

A principal adaptação está no critério de comparação. Um novo caminho substitui o atual quando tem peso menor ou quando tem o mesmo peso e usa menos arestas. A fila de prioridade segue essa mesma ordem: primeiro a distância e depois a quantidade de arestas. Entradas da fila que já foram substituídas por resultados melhores são ignoradas.

Como os pesos são positivos, a distância aumenta a cada aresta percorrida. Isso permite manter a propriedade de Dijkstra: ao retirar da fila um vértice com seu resultado atualizado, esse resultado já é definitivo. O desempate pela quantidade de arestas também é respeitado pela ordem da fila.

Os pesos são representados por BigDecimal para evitar erros de arredondamento nas comparações. Por exemplo, os valores 0.1 + 0.2 e 0.3 precisam ser considerados iguais para que o desempate funcione corretamente. Se a origem for igual ao destino, o resultado é um caminho de peso zero e sem arestas. Quando o destino não é alcançável, o programa informa que não existe caminho.

A implementação ocupa O(V + E) de memória. Como a fila pode manter entradas antigas, o custo de tempo é O(V + E log(E + 1)), desconsiderando o custo variável das operações com BigDecimal. V representa o número de vértices e E, o número de arestas.

3. Entrada e exemplo de execução

O arquivo de entrada começa com o número de vértices e de arestas. Depois, cada linha descreve uma aresta no formato origem, destino e peso. Os vértices são numerados de 1 a n. O nome do arquivo e os vértices da consulta podem ser informados pelo terminal, como em `java Main exemplo.txt 1 6`.

No arquivo de exemplo, existem dois caminhos de peso 4: $1 \to 2 \to 3 \to 6$ e $1 \to 4 \to 6$. O programa retorna o segundo, pois ele tem duas arestas, enquanto o primeiro tem três. A saída mostra o peso total, a quantidade de arestas e a sequência de vértices.

4. Testes e resultados

Foram testados todos os pares de 200 grafos aleatórios, com 2 a 11 vértices e pesos de 1 a 10. Todos os resultados coincidiram com Floyd-Warshall em peso e número de arestas. Também foram conferidos os caminhos e os casos especiais: decimais, laços, arestas paralelas e destinos inacessíveis.

Para avaliar o tempo de execução, foram utilizados dois tipos de grafos, com 100, 200, 400 e 800 vértices. Em todos eles, a consulta foi do vértice 1 ao vértice n. Antes das medições, foram feitas 10 buscas de aquecimento; depois, 21 buscas medidas com System.nanoTime. As tabelas apresentam a mediana dessas medições. O tempo inclui a busca e a reconstrução do caminho, sem leitura do arquivo ou impressão. Os testes foram executados no Windows, com Java 8 (1.8.0_481).

4.1. Grafos esparsos

Neste tipo, cada vértice i se liga a i + 1, com peso 1, e a i + 2, com peso 2, quando esses destinos existem. Assim, o grafo tem 2n − 3 arestas. O peso esperado é n − 1, e o menor número de arestas é o arredondamento para cima de (n − 1)/2. Os resultados da Tabela 1 coincidiram com esses valores em todos os tamanhos.

Tabela 1 – Resultados para o grafo esparso.

Vértices	Arestas	Peso do caminho	Arestas do caminho	Tempo mediano (ms)
100	197	99	50	0,0160
200	397	199	100	0,0741
400	797	399	200	0,0316
800	1597	799	400	0,0406

4.2. Grafos densos

Foram usados grafos direcionados completos, sem laços, com peso |v − u| na aresta de u para v. Nesse caso, há n(n − 1) arestas. O caminho direto de 1 até n tem peso n − 1 e uma aresta. Outros percursos crescentes empatam no peso, mas usam mais arestas. Como mostra a Tabela 2, o programa escolheu a ligação direta nas quatro instâncias.

Tabela 2 – Resultados para o grafo denso.

Vértices	Arestas	Peso do caminho	Arestas do caminho	Tempo mediano (ms)
100	9900	99	1	0,1319
200	39800	199	1	0,3669
400	159600	399	1	0,8866
800	639200	799	1	5,6262

5. Análise dos resultados

As oito instâncias retornaram o peso e o número de arestas esperados. O maior tempo mediano foi de 5,63 ms, no grafo denso de 800 vértices. Nos esparsos, os tempos não cresceram regularmente; por serem buscas curtas, a JVM e a atividade do sistema influenciam bastante. As medições não demonstram, sozinhas, a complexidade do algoritmo.

Essas famílias permitem conferir a resposta e o desempate, mas não representam todos os grafos. Os testes aleatórios complementam a avaliação. As entradas estão em instancias/ e as medições completas em resultados.csv. Para repetir, execute java Testes.